Virtualisation & Cloud

Chapitre 3: conteneurisation

Dr Dhekra EL HAMDI

Contact: hamdi.dhikra@yahoo.fr

2025/2026

Plan

1. Introduction
2. Principe de conteneurisation
3. Conteneurisation VS Virtualisation
4. Avantages des conteneurs Linux
5. Qu’est-ce que Docker ?
6. Architecture Docker, Docker Image, Docker Hub
7. Gestion des Images Docker
8. Gestion des Conteneurs Docker
9. Dockerfile
10. Création d’une image Docker
11. Publication d’une image dans le Docker Hub
12. Persistance des données avec les volumes
13. Gestion de plusieurs conteneurs avec Docker Compose

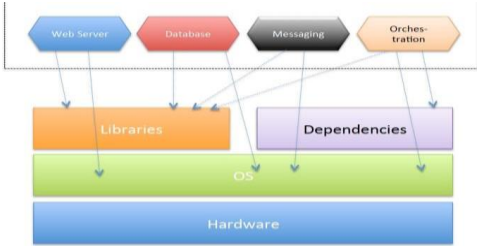
Introduction

Les conteneurs logiciels cherchent à répondre à la même problématique que les conteneurs physiques, ces grandes boites de métal standardisées qui transportent de nos jours l'essentiel des biens matériels par camion, train, avion ou bateau.

Transport de conteneurs physiques : le conteneur est rempli, scellé et transporté sans que le transporteur connaisse le contenu. Standardisé, il permet l'empilement, la logistique optimisée et la sécurité.

Transport de conteneurs logiciels : les applications sont "emballées" avec toutes leurs dépendances. L'infrastructure n'a pas besoin de connaître les détails internes, juste de faire tourner le conteneur intact.
Avantage commun : séparation des responsabilités → fiabilité, standardisation, automatisation et réduction des coûts (physiques ou informatiques).

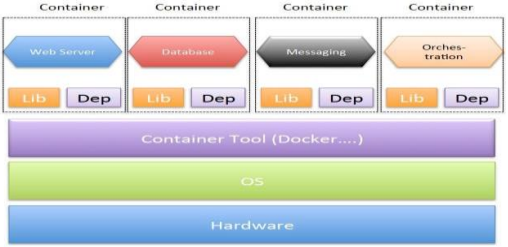
Architecture traditionnelle



- **Problème de compatibilité**
- **Différents environnements : développement, test, production**

➢ **Longue durée d'installation**

Conteneurisation



- Le code est regroupé avec ses **dépendances et configurations** dans un conteneur autonome et **portable**.
- Elle **évite les erreurs** liées aux différences d'environnement (ex. Windows → Linux).

➢ **Exécution fiable sur n'importe quelle plateforme ou cloud**.

Principe de conteneurisation

Un conteneur d'applications est un environnement informatique entièrement regroupé en package et portable :

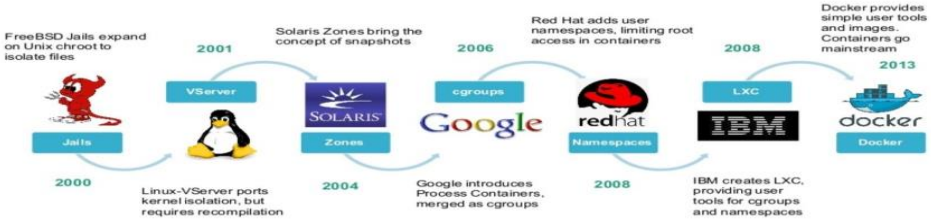
- Il dispose de tout ce dont une application a besoin pour s'exécuter : fichiers binaires, bibliothèques, dépendances et fichiers de configuration, le tout encapsulé et isolé dans un conteneur.
- La conteneurisation d'une application permet d'isoler le conteneur du système d'exploitation hôte, avec un accès limité aux ressources sous-jacentes, à l'instar d'une machine virtuelle légère.

Critères	Conteneurs	Machine Virtuelle (VM)
Isolation	Isolation au niveau du processus et des dépendances à travers des conteneurs. Partage du noyau du système d'exploitation avec l'hôte.	Isolation complète avec un système d'exploitation distinct pour chaque VM.
Système d'exploitation	Les conteneurs partagent le noyau du système hôte, mais peuvent avoir des environnements d'exécution différents (images).	Chaque VM possède son propre système d'exploitation complet, distinct du système hôte.
Performance	Très léger, moins de surcharge car il n'y a pas de virtualisation du matériel. Démarrage et exécution rapides.	Plus lourd, car il y a une virtualisation complète du matériel. Le démarrage et l'exécution sont plus lents que conteneur.
Utilisation des ressources	Consomme moins de ressources (CPU, RAM) car il n'y a pas de duplication de l'OS.	Consomme plus de ressources en raison de la nécessité de virtualiser tout un système d'exploitation.

Démarrage	Démarrage en quelques secondes, car il n'y a pas besoin de charger un OS complet.	Plus lent à démarrer car le système d'exploitation de la VM doit être démarré.
Compatibilité	Dépend du système d'exploitation hôte, les conteneurs Linux ne peuvent pas fonctionner directement sur un hôte Windows (et vice-versa).	Indépendante du système d'exploitation hôte, une VM peut exécuter n'importe quel système d'exploitation (Windows, Linux, etc.).
Portabilité	Très portable, les images Docker peuvent être déployées sur n'importe quel environnement compatible Docker.	Moins portable que les conteneurs, nécessite des hyperviseurs et des configurations spécifiques pour chaque plateforme.
Sécurité	L'isolation est moins stricte comparée aux VMs, bien que des outils puissent renforcer la securité.	Forte isolation, chaque VM étant complètement séparée avec son propre OS.

Un Conteneur Linux, qu'est-ce que c'est ?

- Les conteneurs Linux **LXC** (Linux Container) existent depuis plus longtemps que **Docker**



- Les technologies de conteneurs Linux offrent une solution de packaging pour une application et ses dépendances.

- Les conteneurs sont isolés du reste du système. Ils sont packagés en un ensemble cohérent et prêt à être déployé sur un serveur et son OS

Qu’est ce que Docker ?

« Build, Ship and Run Any App, Anyhere »

Qu’est ce que Docker ?

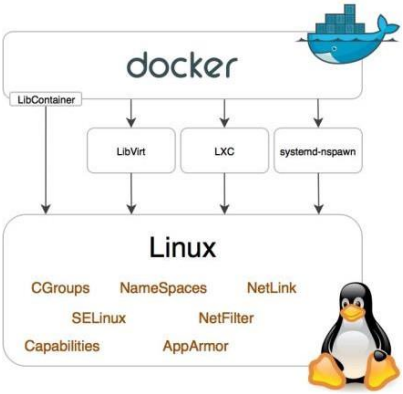
- **Docker** est un projet open source créé par la société **Dotcloud** (devenue ensuite *Docker Inc.*).

  Il repose sur la technologie des **conteneurs Linux (LXC)**.
- C’est un **outil** qui permet de **regrouper une application et toutes ses dépendances** dans un **conteneur**.

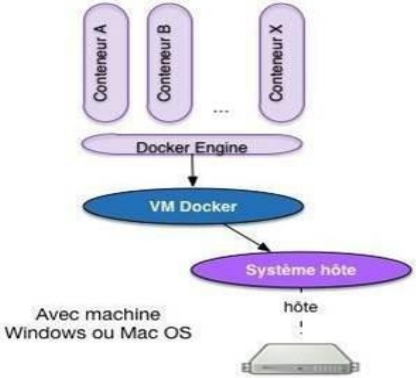
  Ce conteneur peut être exécuté **sur n’importe quel serveur Linux**, et même sur **Windows** ou **Mac**, car Docker améliore la technologie LXC.
- Docker peut aussi être **utilisé dans le Cloud**, c’est pourquoi de grandes entreprises comme **Google, Microsoft** ou **Amazon** l’ont intégré dans leurs services.

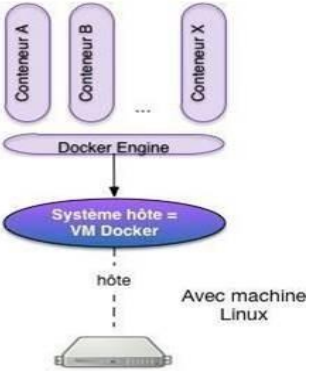
Qu’est ce que Docker ?

- Depuis la version 0.9 (2014), Docker n’utilise plus **LXC** comme environnement d’exécution par défaut.
- Il a été remplacé par **Libcontainer**, une bibliothèque développée en **Go**, qui offre une meilleure abstraction du système d’exploitation hôte.
- Cette évolution permet à **Docker** de fonctionner sur d’autres systèmes que Linux, comme **Windows**, **Mac**, **BSD** ou **Solaris**, ce qui facilite le **déploiement continu**.



L'architecture Docker, sous différents systèmes d'exploitation





Installation Docker

❖ sous Linux

```
$ curl -fsSL https://get.docker.com -o get-docker.sh
$ sudo sh get-docker.sh
```

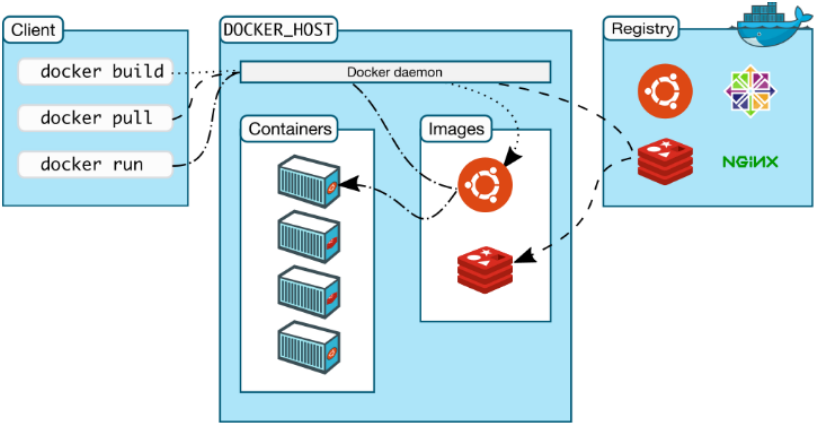
❖ sous Windows

Télécharger, installer l'outil « Docker Desktop Installer » à partir du site officiel et se connecter avec le compte créé.

Architecture Docker

- Docker permet aux utilisateurs de démarrer, d'arrêter et de gérer les conteneurs logiciels directement à partir du terminal (CLI)

Docker Engine



Docker Engine

Le moteur Docker repose sur une **architecture client-serveur** composée de trois éléments principaux : **Docker Daemon**, **Docker CLI** et **l'API Docker**, qui sert d'interface entre les deux.

- **Docker Daemon** : il représente la **partie serveur** du moteur Docker.
  Exécuté en arrière-plan sur le système hôte, il gère de manière centralisée la création et l'administration des **images**, **conteneurs** et **réseaux** Docker.
  Il réagit aux commandes telles que build (créer), pull (télécharger) ou run (exécuter).
- **API Docker** : elle repose sur le modèle **REST** et permet de communiquer avec le daemon en lui indiquant les actions à effectuer.
- **Docker CLI (Command Line Interface)** : c'est l'interface de communication entre le daemon et l'utilisateur final via l'API REST. Celui-ci peut être installé sur un client physique distinct du serveur exécutant le Docker Daemon.

Architecture Docker

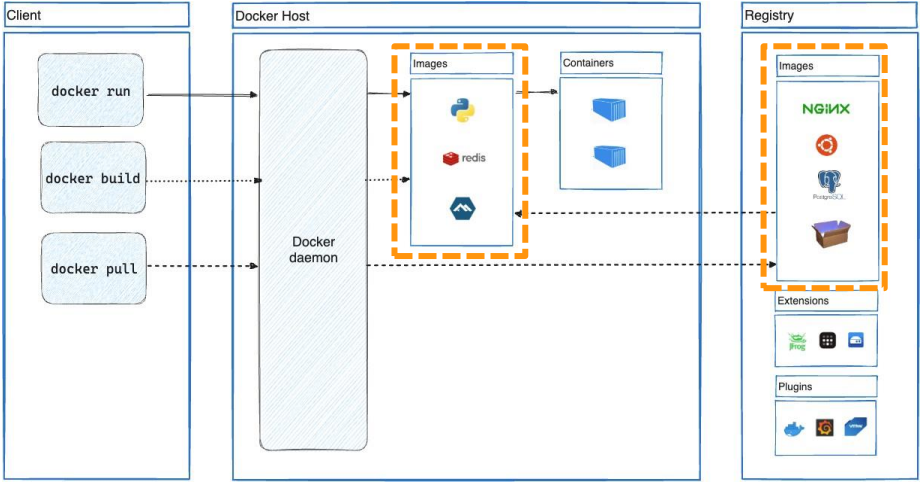


Image Docker

Les images de conteneurs

- Package de l'application et de ces dépendances
- Peut être exécutée sur différents environnements
- Décrites par un fichier texte.
- Infrastructure-as-code

Un Conteneur

- Une instance d'une image de conteneur
- S'exécute dans un environnement isolé

Analogie POO

- Une image = une classe
- Un conteneur = une instance

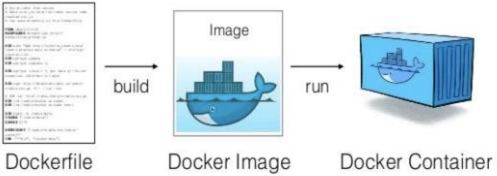


Image Docker

Images Docker vs Images de VM

Les images de machines virtuelles

- Sauvegarde de l'état de la VM (Mémoire, disques virtuels, etc) à un moment donné
- La VM redémarre dans l'état qui a été sauvegardé

Les images Docker

- Une copie d'une partie d'un système de fichier
- Pas de notion d'état
- Un ensemble de couches:
    - Une image est composée d'un ensemble de couches (layers)
    - L'Union File System est utilisé pour combiner ces couches
    - Le system utilisé par défaut s'appelle overlay2
    - Chaque couche correspond à une instruction dans le fichier Dockerfile décrivant l'image

Docker Hub

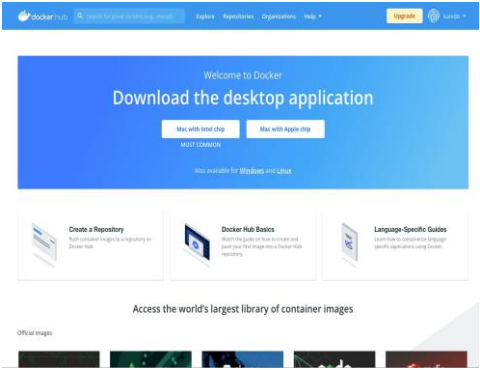
- Le registre Docker permet de télécharger, partager, indexer des images Docker en respectant les standards de publications des référentiels Git. Docker dispose d'un référentiel central public appelé Docker Hub

- Le Docker Hub est un registre basé sur le Cloud, c'est une sorte de bibliothèque pour les images Docker.
  - un store ou les utilisateurs de Docker peuvent partager leurs images.
- Plusieurs images sont disponibles (des images de base et des images préconfigurées)
  - Les images de base officielles ont été créées par l'équipe de développement de Docker.
- Le service en ligne est divisé en un espace public et un espace privé
  - L'espace public offre aux utilisateurs la possibilité de télécharger leurs propres images et de les partager avec la communauté.
  - Les images téléchargées dans une zone privée du registre ne sont pas accessibles au public et peuvent, par exemple, être partagées au sein de l'entreprise ou avec des amis et des connaissances.

Docker Hub

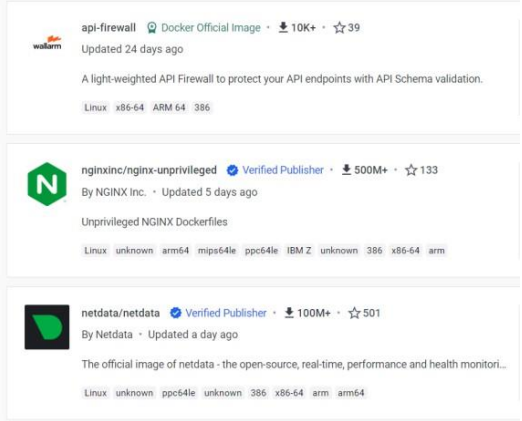
- Un serveur stockant des images docker
- Possibilité de récupérer des images depuis ce serveur (pull)
- Possibilité de publier de nouvelles images (push)

- Il est accessible via : https://hub.docker.com/



Docker Hub

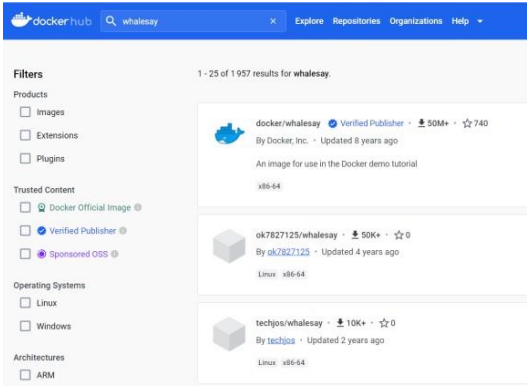
- Le nom de l'image est généralement sous la forme USER/IMAGE_NAME, sauf dans le cas d'une image officielle, où c'est seulement IMAGE_NAME
- Le nombre de stars : Le système de notation
- Le nombre de pulls : Le nombre de téléchargements

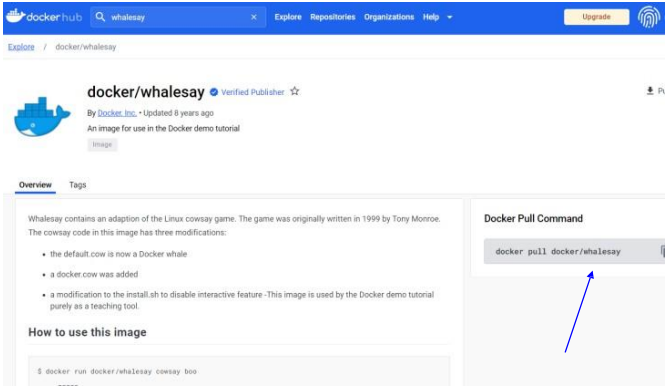


Gestion des Images Docker

- **Télécharger une image Docker** : commande docker pull IMAGE

Sur la page d'accueil de Docker Hub, on trouve par exemple l'image *whalesay* (docker demo) en tapant, on peux cliquer sur la ressource docker/whalesay pour accéder au Repository public de cette image.





Gestion des Images Docker

Afficher toutes les images du Docker sur votre système local

$ docker image ls

- La commande docker image donne tout sur les images locales, y compris la taille du fichier, la balise et l'ID de l'image

```
$ docker images
REPOSITORY                  TAG            IMAGE ID       CREATED         SIZE
jekyll-github-page          latest         65736152d458   4 days ago      735MB
maven                       3.8.1-jdk-11   4e8fadf07a8c   7 months ago    658MB
openjdk                     11-jdk         a9c7e4ad6720   8 months ago    647MB
ubuntu                      20.04          7e0aa2d69a15   8 months ago    72.7MB
```

- REPOSITORY : nom de l'image.
- TAG : sont utilisés principalement pour affecter une version à une image
- ID : L'identifiant de l'image (unique pour chaque image téléchargée)
- CREATED : Date de la dernière modification de l'image
- SIZE : Taille de l'image

Gestion des Images Docker

Supprimer une image Docker

$ docker image rm [nom ou ID Image]:[tag]

- Une image Docker ne peut pas être supprimée si des conteneurs ont été instanciés depuis cette image (ça va aussi supprimer nos conteneurs qui se basent sur cette image)
- On peut forcer la suppression pour éliminer aussi les conteneurs liés à l'image
- **$ docker image rm –f [nom ou ID Image]:[tag]**

Gestion des Conteneurs Docker

Démarrer une image Docker en tant que conteneur

$ docker run [OPTIONS] [nomImage]:[tag]

- La seule partie obligatoire est le nom de l'image Docker désiré
- Exemple d'option
  - **-d** (abréviation --detach) : pour exécuter le conteneur en arrière plan. Il permet de continuer à utiliser la console pendant que le conteneur tourne;
  - **-p** (abréviation --publish) pour définir l'utilisation de ports. La commande crée un mappage de port entre l'hôte et le conteneur.
    - Sans le mappage des ports, vous ne pourriez pas accéder à l'application depuis l'hôte

Gestion des Conteneurs Docker

Démarrer une image Docker en tant que conteneur

- Démarrons Nginx (serveur HTTP + reverse proxy) et exposons le port 80 du conteneur sur le port 8000 de notre machine:

```
$ docker run -d -p 8000:80 nginx
```

Gestion des Conteneurs Docker

Démarrer une image Docker en tant que conteneurs

- Lorsque vous démarrez un conteneur, l'image sous-jacente est téléchargée sous forme de copie à partir du référentiel et stockée de façon permanente sur votre ordinateur.
- Un nouveau téléchargement n'est lancé que si la source de l'image change, par exemple si une version plus récente est disponible dans le référentiel.

```
hp@hp-HP-Laptop-15-dw3xxx:~/Bureau$ docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
578acb154839: Pull complete
e398db710407: Pull complete
85c41ebe6d66: Pull complete
7170a263b582: Pull complete
8f28d06e2e2e: Pull complete
6f837de2f887: Pull complete
c1dfc7e1671e: Pull complete
Digest: sha256:86e53c4c16a6a276b204b0fd3a8143d86547c967dc8258b3d47c3a2
Status: Downloaded newer image for nginx:latest
5e859f94fbf99c3e749104464f7a34fdab1e3aa63df7c51f2486c32efc579dfb
```

Gestion des Conteneurs Docker

```
$ docker run hello-world
```

Que va-t-il se passer?

- L'image à charger est hello-world:latest
- Vérification: est ce que l'image est présente localement?

Sinon, télécharger l'image depuis Docker Hub

- Charger l'image dans le conteneur et exécuter la commande par défaut dénie pour ce conteneur

Gestion des Conteneurs Docker

Afficher tous les conteneurs exécutés sur le système

$ docker ps -a

- La sortie du terminal comprend des informations telles que l'ID du conteneur respectif, l'image sous-jacente, la commande exécutée lorsque le conteneur a été démarré, l'heure à laquelle le conteneur respectif a été démarré et l'état actuel.

```
hp@hp-HP-Laptop-15-dw3xxx:~/Bureau$ docker ps -a
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS
5e859f94fbf9   nginx            "/docker-entrypoint.…"   17 seconds ago   Up 15 seconds
f8214b28518f   getting-started  "docker-entrypoint.s…"   4 days ago       Created
f20f3e9b8fa6   hello-world      "/hello"                 5 days ago       Exited (0) 5 days ago
1b40e1b617de   c1087b2b41a9     "docker-entrypoint.s…"   3 months ago     Exited (0) 3 months ago
24c8eb35faa8   c1087b2b41a9     "docker-entrypoint.s…"   3 months ago     Created
485e5ec04644   hello-world      "/hello"                 3 months ago     Exited (0) 3 months ago
60af50e0659d   hello-world      "/hello"                 3 months ago     Exited (0) 3 months ago
```

Gestion des Conteneurs Docker

- Stopper un conteneur Docker

  $docker stop (Nom ou ID du conteneur)

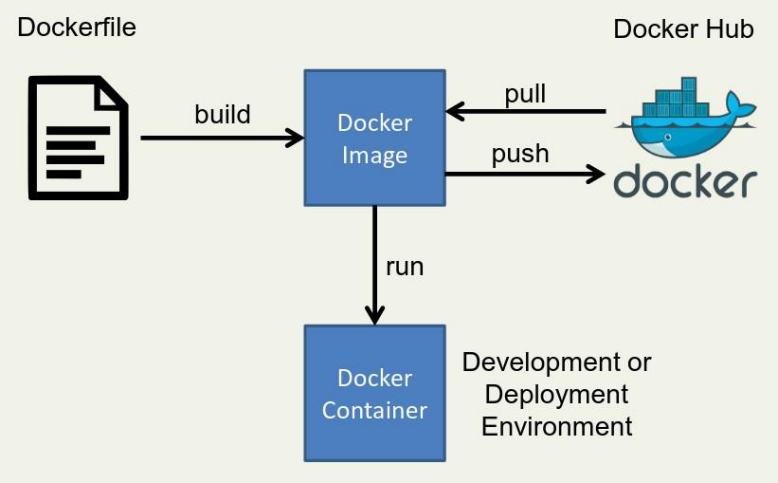
- Démarrer un conteneur stoppé

  $docker start (Nom ou ID du conteneur)

- Supprimer un conteneur stoppé

  $docker rm (Nom ou ID du conteneur)

Dockerfile


Dockerfile
Docker Hub
build
Docker Image
pull
push
docker
run
Docker Container
Development or Deployment Environment

Dockerfile

- Le Dockerfile est un fichier qui contient toutes les instructions et arguments pour créer une image, comme les commandes à exécuter pour installer un logiciel, etc.
- Un Dockerfile est une recette décrivant comment construire une image
- Contient une suite d'instructions
- La commande docker build permet de créer une image à partir d'un Dockerle

Dockerfile

- Exemple d’un Dockerfile

Instruction	Argument

```
# Utiliser une image de base

FROM Ubuntu

# Définir le répertoire de travail

WORKDIR /app

# Mettre à jour le répertoire apt et Installer les dépendances

RUN apt-get update
RUN apt-get install python

RUN pip install flask
RUN pip install flask-mysql

# Copier le code de l'application

COPY . .

# Définir la commande de démarrage de l’application

CMD ["python", "app.py"]
```

Dockerfile

FROM: Définit l'image à partir de laquelle la nouvelle image est créée
LABEL: Associe des meta-données à la nouvelle image (par exemple, l'auteur de l'image)
RUN: Dénie une commande exécutée dans la couche au dessus de l'image courante lors de la construction de l'image
CMD: Dénie la commande exécutée au démarrage du conteneur
EXPOSE: Informe docker que le conteneur va écouter sur le port réseau déni
COPY: Copier un fichier/répertoire depuis le contexte de construction de l'image vers la nouvelle couche

- La destination peut être un chemin absolu ou un chemin relatif depuis **WORKDIR**
- Les commandes de Dockerfile ne sont pas sensibles à la casse. On les note en majuscule par convention (facilite la lecture)

Couches d’une image Docker

Images de base

- Toute image est définie à partir d'une image de base
- Des images de base officielles sont déjà fournies
- Exemples: ubuntu:latest, ubuntu:14.04, opensuse:latest, alpine:latest

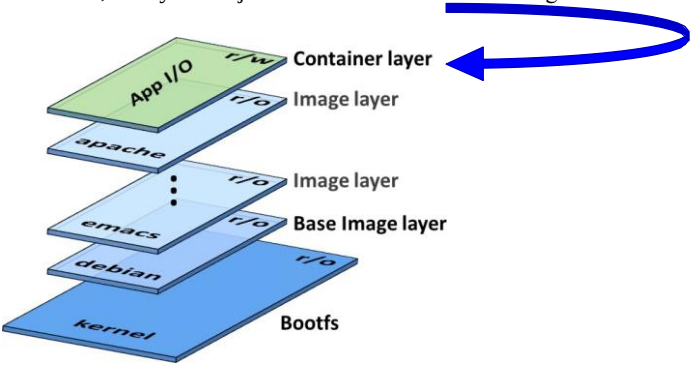
alpine: Image de base minimaliste (5MB) -- intéressant pour les performances

Relation avec le système d'exploitation hôte

- Une image n'inclut que les bibliothèques et services du système d'exploitation mentionné
- Le conteneur utilise le noyau du système hôte

Couches d'une image Docker

- Une image docker est une succession de couches (layers) qui contiennent une liste de modifications du système de fichiers
  - On peut faire une analogie avec GIT où chaque layer serait un commit
- Au moment de la création d'un conteneur, un layer est ajouté au-dessus de ceux de l'image



Couches d’une image Docker

- Les couches correspondent aux différentes modifications qui sont faites pour construire l'image à partir de l'image de base.

  - ❖ Pour sauvegarder une nouvelle image, il suffit de sauvegarder les nouvelles couches qui ont été créées au dessus de l'image de base
  - ❖ Chaque couche capture les écritures faites par une commande exécutée lors de la création de l'image
  - ❖ Faible espace de stockage utilisé

Dockerfile

- Chaque **instruction** va créer une nouvelle **layer** correspondant à chaque **étape** de la construction de l'image, ou de la recette.

```
# Utiliser une image de base
FROM Ubuntu

# Définir le répertoire de travail
WORKDIR /app
# Mettre à jour le répertoire apt et Installer les dépendances
RUN apt-get update
RUN apt-get install python

RUN pip install flask
RUN pip install flask-mysql

# Copier le code de l'application
COPY . .
# Définir la commande de démarrage de l'application
CMD ["python", "app.py"]
```

Layer 1 : Base Ubuntu Layer

Layer 2 : Working directoty

Layer 3 : Changes in apt package

Layer 4 : Changes in pip package

Layer 5 : Source code

Layer 6 : Default command when running the container

Création de votre propre image Docker

Création de votre propre Dockerfile

- Nous allons utiliser le code de l'application ‘’To Do’’ (application Node.js) :

  https://github.com/docker/getting-started-app/tree/main

- Nous allons créer le Dockerfile et le mettre dans le répertoire racine du projet

```
#image source
FROM node:18-alpine

#définir le répertoire de travail
WORKDIR /app

# copier le code source
COPY . .

#installer les dépendances
RUN yarn install --production

#définir la commande par défaut lors de l’exécution de vos conteneurs Docker
CMD ["node", "src/index.js"]

#définir les ports d'écoute par défaut
EXPOSE 3000
```

RUN yarn install --production

Cette commande installe uniquement les **dépendances nécessaires au fonctionnement** de l’application Node.js, celles listées dans package.json.

•**yarn** est un gestionnaire de paquets, similaire à **npm**.

•L’option --production signifie qu’on **ignore les dépendances de développement**, pour ne garder que celles utiles en production.

Création de votre propre image Docker

Création de votre propre Dockerfile

- Création de l'image Docker

docker build -t getting-started .

L'option –t permet de marquer l'image

Signifie que le Dockerfile se trouve dans le répertoire courant

Création de votre propre image Docker

docker build -t getting-started .

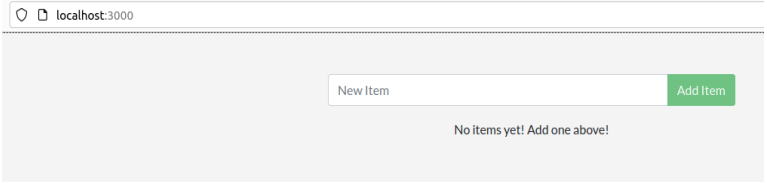
```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker build -t getting-started .
[+] Building 54.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 466B
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load metadata for docker.io/library/node:18-alpine
 => [auth] library/node:pull token for registry-1.docker.io
 => [1/4] FROM docker.io/library/node:18-alpine@sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d
 => => resolve docker.io/library/node:18-alpine@sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d
 => => sha256:d1517ab6615b7b1f3b81f339100063d1b2b41f1a32a9efb8563ecd1375311c22 6.78kB / 6.78kB
 => => sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa 3.40MB / 3.40MB
 => => sha256:3130715204cf4a9be94608d180505f50862416589a8f03eba7b664f15b9c0283 47.88MB / 47.88MB
 => => sha256:b06de8ab1c4feccaf7b687bb7ebd5180c2bd1f59d91749619d52af77fd38ea13 2.34MB / 2.34MB
 => => sha256:435dcad253bb5b7f347ebc69c8cc52de7c912eb7241098b920f2fc2d7843183d 1.43kB / 1.43kB
 => => sha256:51490771aba658439d29b1b03b60fc31e67bf0da3e01cb5903716310df4be1c1 1.16kB / 1.16kB
 => => sha256:90ef3ffc51561ffa7dfafd2dc93f44601f8d4d4273ad8d54dbf34326c746a142 448B / 448B
 => => extracting sha256:96526aa774ef0126ad0fe9e9a95764c5fc37f409ab9e97021e7b4775d82bf6fa
 => => extracting sha256:3130715204cf4a9be94608d180505f50862416589a8f03eba7b664f15b9c0283
 => => extracting sha256:b06de8ab1c4feccaf7b687bb7ebd5180c2bd1f59d91749619d52af77fd38ea13
 => => extracting sha256:90ef3ffc51561ffa7dfafd2dc93f44601f8d4d4273ad8d54dbf34326c746a142
 => [internal] load build context
 => => transferring context: 4.59MB
 => [2/4] WORKDIR /app
 => [3/4] COPY . .
 => [4/4] RUN yarn install --production
 => exporting to image
 => => exporting layers
 => => writing image sha256:4ac73a4be5d6e114a9c8e7d6c8c2fc28002c4d98d0214a3ad3d43ed612f13e82
 => => naming to docker.io/library/getting-started
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Création de votre propre image Docker

Démarrer votre Conteneur **docker run -dp 3000:3000 getting-started**

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS          PORTS                                      NAMES
649808215840   getting-started  "docker-entrypoint.s…"   40 seconds ago   Up 39 seconds   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp  affectionate_diffie
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Navigateur de la machine hôte :



Création de votre propre image Docker

1. **Mettre à jour l'application**
   Modifier le fichier src/static/js/app.js (par exemple, à la ligne 56) pour changer le message "No items yet! Add one above!".

2. **Recréer l'image Docker**
   Construire une nouvelle version de l'image avec : docker build -t getting-started .

3. **Lancer le conteneur mis à jour**
   Démarrer un nouveau conteneur utilisant le code modifié : docker run -dp 3000:3000 getting-started

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker run -dp 3000:3000 getting-started
f8214b28518f3a59ae6445dccf9e0e37e99a0c8e25159761ae104d236e74525e
docker: Error response from daemon: driver failed programming external connectivity on endpoint mystifying_mcnulty (481596
1282c3bec4307422e80dbf3ce0a541c14ca0e680e99bc8bed4f2531704): Bind for 0.0.0.0:3000 failed: port is already allocated.
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

- Une erreur s'affiche parce que l'ancien conteneur utilise déjà le port 3000 de l'hôte et un seul processus sur la machine (conteneurs inclus) peut écouter un port spécifique. Pour résoudre ce problème, vous devez supprimer l'ancien conteneur puis démarrer le conteneur de l'application mise à jour

Création de votre propre image Docker

Mettre à jour votre application

- **Question :** Peut-on modifier le code et voir les changements immédiatement, sans recréer l'image Docker ?
- **Réponse :** Oui ! Il suffit d'utiliser un **Bind Mount** (ou mappage de répertoire).

Bind mount : permet de **partager un dossier de l'hôte avec le conteneur**.

Cela permet de **monter le code source** dans le conteneur. Ainsi, toute modification enregistrée sur l'hôte est immédiatement visible dans le conteneur.

Vous pouvez exécuter des processus dans le conteneur qui **surveillent les fichiers** et réagissent aux changements.

Exemple : pour une application Node.js, l'outil **nodemon** surveille les fichiers et **redémarre automatiquement** l'application lorsque des modifications sont détectées. Des outils similaires existent pour la plupart des autres langages et frameworks.

Application

- Créer un `Dockerfile` pour une application `Node.js` dans un répertoire `my-app` avec les fichiers `index.js` (qui contient le code source de l'application) et `package.json` (pour les dépendances de l'application). L'application écoute sur le port `8080`.

node

Updated 40 minutes ago

Node.js is a JavaScript-based platform for server-side and networking applications.

LANGUAGES & FRAMEWORKS

How to use this image

See How To Use This Image on GitHub for up-to-date documentation.

Image Variants

The `node` images come in many flavors, each designed for a specific use case.

`node:<version>`

This is the defacto image. If you are unsure about what your needs are, you probably want to use this one. It is designed to be used both as a throw away container (mount your source code and start the container to start your app), as well as the base to build other images off of.

Some of these tags may have names like bookworm or bullseye in them. These are the suite code names for releases of Debian and indicate which release the image is based on. If your image needs to install any additional packages beyond what comes with the image, you'll likely want to specify one of these explicitly to minimize breakage when there are new releases of Debian.

This tag is based off of `buildpack-deps`. `buildpack-deps` is designed for the average user of Docker who has many images on their system. It, by design, has a large number of extremely common Debian packages. This reduces the number of packages that images that derive from it need to install, thus reducing the overall size of all images on your system.

`node:<version>-alpine`

This image is based on the popular Alpine Linux project, available in the `alpine` official image. Alpine Linux is much smaller than most

Portabilité d'une image Docker

- Une image Docker créée sur un système d'exploitation (Linux ou Windows) **ne peut pas toujours être utilisée directement** sur l'autre système sans quelques ajustements.
- Les images sont composées de **couches** qui dépendent du **système de fichiers spécifique** à chaque plateforme.

 Build « Multi-platformes »

Portabilité d'une image Docker

Build « Multi-platformes »

- Le moyen le plus simple de commencer à créer des images pour plusieurs plates-formes est d'utiliser l'émulation. Avec l'émulation, vous pouvez créer votre application sur plusieurs architectures sans avoir à apporter de modifications à votre Dockerfile.
- Il suffit de transmettre l'indicateur --platform à la commande build, en spécifiant le SE et l'architecture pour lesquels vous souhaitez créer l'image
- La commande ci-dessous crée l'image du serveur pour la plateforme Linux/arm/v7 :
  - docker build -t=server --platform=linux/arm/v7 .

Création de votre propre image Docker

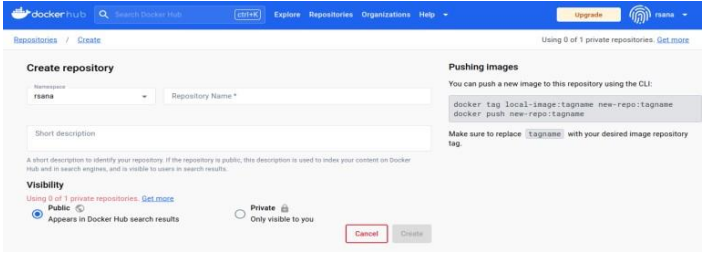
Build « Multi-platformes »

- Il est possible d'utiliser l'émulation pour produire des sorties pour plusieurs plates-formes à la fois.
  - Ex : --platform=linux/amd64,linux/arm/v7,linux/arm64/v8
- Mais, le pilote de build par défaut ne prend pas en charge les build multiplateformes simultanées. Donc, d'abord, on doit utiliser un autre générateur, qui utilise un pilote prenant en charge les versions multiplateformes simultanées
- Docker Buildx
  - Buildx est le client de build de nouvelle génération et il offre une expérience utilisateur similaire à la commande docker build classique, tout en prenant en charge des fonctionnalités supplémentaires.

Publier une image dans le Docker Hub

- Vous pouvez partager votre image auto-créée dans le Docker hub pour la rendre disponible à la communauté ou à un groupe de travail
- Un Docker ID est nécessaire pour accéder au Docker Hub
- Etapes nécessaires pour publier l'image:

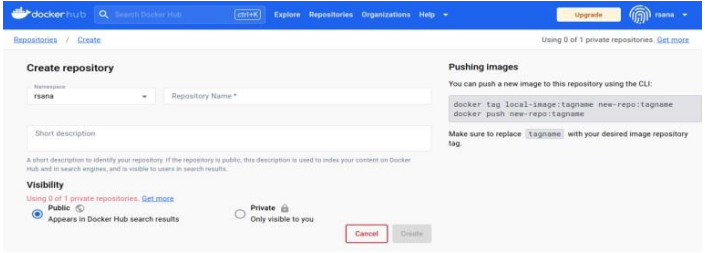
Étape 1. Créer un repository : connectez-vous au Docker Hub avec votre ID Docker et votre mot de passe personnel et créez un repository public nommé getting-started (le même nom que l'image)



Publier une image dans le Docker Hub

- Vous pouvez partager votre image auto-créée dans le Docker hub pour la rendre disponible à la communauté ou à un groupe de travail
- Un Docker ID est nécessaire pour accéder au Docker Hub
- Etapes nécessaires pour publier l'image:

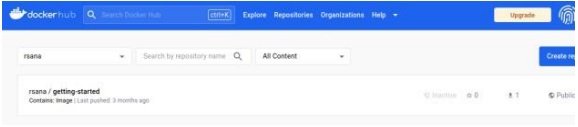
Étape 1. Créer un repository : connectez-vous au Docker Hub avec votre ID Docker et votre mot de passe personnel et créez un repository public nommé getting-started (le même nom que l'image)



Publier une image dans le Docker Hub

Étape 3. Publier une Image

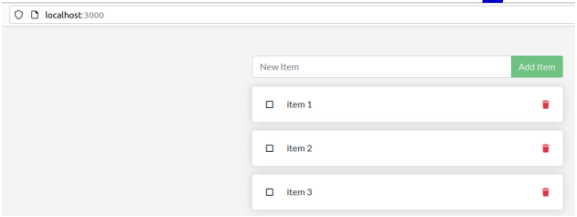
- Il faut se connecter au Docker Hub avec la commande docker login : $ sudo docker login
  - Le terminal vous invite à entrer votre nom d'utilisateur (Docker ID) et votre mot de passe.
- Si la connexion a réussi, utilisez la commande docker push pour publier votre image vers le Repository nouvellement créé.
  - $ sudo docker push myUserName/getting-started
- Connectez-vous au Docker-Hub via votre navigateur pour voir l'image téléchargée.

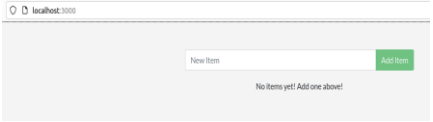


Publier une image dans le Docker Hub

- Pour cette application « To Do » (dont l'image Docker est intitulée « getting-started »), remarquez que votre liste de tâches est vide à chaque fois que vous lancez le conteneur

Redémarrage de l'instance du conteneur



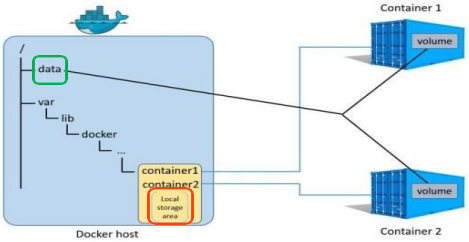


Persistance des données avec les volumes

- Par défaut, l'application **« To Do »** enregistre ses données dans une base **SQLite**, sous forme du fichier todo.db situé dans le système de fichiers du conteneur.
- Si le conteneur est supprimé, toutes les données qui **ne sont pas dans un volume** disparaissent avec lui.
- En créant un **volume** et en le **connectant** (ou « montant ») au dossier contenant les données, celles-ci sont **conservées sur l'hôte**. Ainsi, lorsque le conteneur écrit dans todo.db, les informations restent **persistantes** dans le volume, même après la suppression du conteneur.

Persistance des données avec les volumes

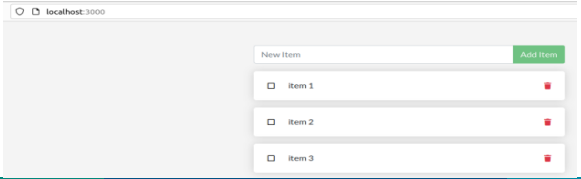
- Le schéma montre un **hôte Docker** avec **deux conteneurs** en fonctionnement.
- Chaque conteneur possède son **espace de stockage propre** dans la zone locale de l'hôte (/var/lib/docker/...).
- À côté de cela, un **volume de données partagé** est situé sur l'hôte, par exemple à /data.
- Les volumes sont **indépendants des conteneurs** : même si un conteneur est supprimé, les données qu'il contient dans le volume restent **disponibles sur l'hôte**.



Persistance des données avec les volumes

Créer un **volume** nommé **todo-db** à l'aide de la commande :: docker volume create todo-db

- Arrêter puis supprimez le conteneur de l'application qui fonctionnait sans volume persistant : docker rm -f <id-conteneur>
- Relancer ensuite le conteneur en ajoutant l'option **--mount** pour associer le volume.
  Par exemple, montez le volume sur le chemin **/etc/todos** dans le conteneur afin d'y stocker les fichiers créés: docker run -dp 3000:3000 --mount type=volume,src=todo-db,target=/etc/todos getting-started
- Vérifier enfin que les données sont bien **persistantes**, même après un **redémarrage** ou la **suppression** du conteneur.



Persistance des données avec les volumes

Un **volume Docker** est un espace de stockage externe au conteneur, utilisé pour **conserver les données de manière persistante**, même après la suppression du conteneur.

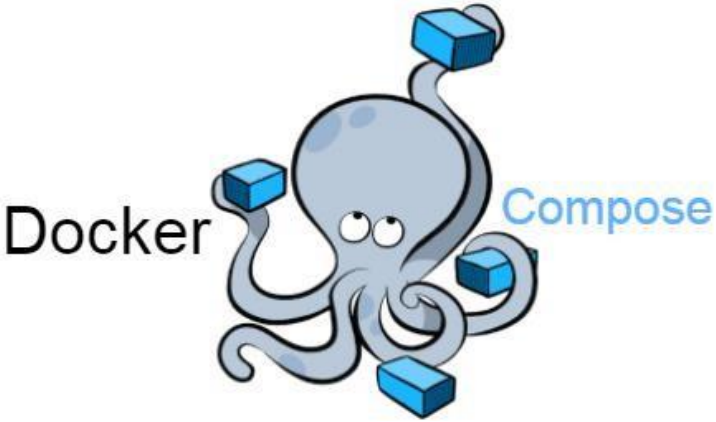
Il permet une **séparation nette entre l'application et ses données**, et peut être **partagé entre plusieurs conteneurs** en même temps.

Les volumes sont **gérés automatiquement par Docker**, qui se charge de leur emplacement sur le disque (souvent dans /var/lib/docker/volumes/).

L'utilisateur n'a besoin de retenir que le **nom du volume**.

- **Lister les volumes :** docker volume ls
- **Supprimer un volume :** docker volume rm nom_volume

Gestion de plusieurs conteneurs avec Docker



Docker Compose

- Supposons qu'on veut ajouter une base de données MySQL à la pile de notre application « To Do »
- Question : est ce qu'on installe MySQL dans le même conteneur ou on l'exécute séparément
  - Réponse : en général, chaque conteneur doit faire une seule chose et bien la faire
- Dans la vision Docker, un conteneur ne doit faire tourner qu'un seul processus

Docker Compose

Défis

- Applications multi-conteneurs: une application Docker qui inclut plusieurs conteneurs (par exemple, un serveur Web et une base de données s'exécutant dans des conteneurs distincts)
- La construction, l'exécution et la connexion des conteneurs à partir de fichiers Docker distincts sont fastidieuses

➢ **Docker Compose** un outil permettant de définir (dans un fichier YAML), et de gérer (en ligne de commande) plusieurs conteneurs comme un ensemble de services interconnectés

- Fichier YAML déclaratif: docker-compose.yml (placé à la racine du projet)
- exécuter une seule commande pour créer, exécuter et configurer tous les conteneurs décrits dans le fichier YAML.

Docker Compose

Les **principals sections** d'un fichier docker-compose.yml :

- **services** (les conteneurs à exécuter) : cette section définit les services qui composent l'application. Un service peut être une application, une base de données, etc. Chaque service est configuré pour être exécuté dans un conteneur Docker.

```
services:
  app:
    image: node:18-alpine
    ports:
      - "3000:3000"
```

Le nom du premier service

- **volumes** : utilisés pour persister les données entre les redémarrages des conteneurs ou pour partager des données entre plusieurs services.
- **networks** : permettent aux conteneurs de communiquer entre eux. Docker Compose crée par défaut un réseau, mais on peux définir un ou plusieurs réseaux en cas de besoin.

Docker Compose

- Après avoir définir un volume dans la section « volumes » (même niveau que les services), on peux utiliser ce volume dans un service :

```
services:
  app:
    image: node:18-alpine
    ports:
      - "3000:3000"
  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data :/var/lib/mysql

volumes:
  todo-mysql-data :
```

- De même, une fois un réseau est définit dans la section « networks » (même niveau que les services), on peux lier les services a ce réseau :

```
services:
  app:
    image: node:18-alpine
    networks:
      - mynetwork
  mysql:
    image: mysql:8.0
    networks:
      - mynetwork
networks:
  mynetwork
  :
```

Docker Compose : Exemple

(Suite)

```
version: "3.8"                    ← Section 1

services:                         ← Section 2
  app:                            ← Service 1
    image: node:18-alpine
    working_dir: /app
    ports:
      - "3000:3000"
    environment:
      - MYSQL_HOST=mysql
      - MYSQL_USER=root
      - MYSQL_PASSWORD=my-secret-pw
      - MYSQL_DATABASE=todo_db
```

```
  mysql:                          ← Service 2
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: my-secret-pw
      MYSQL_DATABASE: todo_db  # Création de la base de données au démarrage
    volumes:
      - todo-mysql-data:/var/lib/mysql

volumes:                          ← Section 3
  todo-mysql-data:
```

Docker Compose: Exemple

L'image Mysql dispose de plusieurs variables d'environnements

Pour en savoir plus sur les variables d'environnement : Mysql Docker Hub listing

Environment Variables

When you start the `mysql` image, you can adjust the configuration of the MySQL instance by passing one or more environment variables on the `docker run` command line. Do note that none of the variables below will have any effect if you start the container with a data directory that already contains a database: any pre-existing database will always be left untouched on container startup.

See also https://dev.mysql.com/doc/refman/5.7/en/environment-variables.html for documentation of environment variables which MySQL itself respects (especially variables like `MYSQL_HOST`, which is known to cause issues when used with this image).

`MYSQL_ROOT_PASSWORD`

This variable is mandatory and specifies the password that will be set for the MySQL `root` superuser account. In the above example, it was set to `my-secret-pw`.

`MYSQL_DATABASE`

This variable is optional and allows you to specify the name of a database to be created on image startup. If a user/password was supplied (see below) then that user will be granted superuser access (corresponding to `GRANT ALL`) to this database.

`MYSQL_USER`, `MYSQL_PASSWORD`

These variables are optional, used in conjunction to create a new user and to set that user's password. This user will

Lancer la stack Docker Compose

Démarrer l'ensemble des conteneurs en arrière plan

docker compose up -d

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker compose up -d
[+] Running 17/17
 ✔ mysql 11 layers [⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿⣿]      0B/0B      Pulled
   ✔ 8e0176adc18c Pull complete
   ✔ a6b6bf6e5d0f Pull complete
   ✔ c17b83f8620f Pull complete
   ✔ b2e259cd9b6c Pull complete
   ✔ 366131ab00d1 Pull complete
   ✔ 2f99ba83a3cb Pull complete
   ✔ f7c88955f01f Pull complete
   ✔ 577fb415d7f8 Pull complete
   ✔ 29160ed46eb1 Pull complete
   ✔ 69ce9884ce5d Pull complete
   ✔ 848f0dceb14c Pull complete
 ✔ app 4 layers [⣿⣿⣿⣿]      0B/0B      Pulled
   ✔ 96526aa774ef Already exists
   ✔ 3130715204cf Already exists
   ✔ b06de8ab1c4f Already exists
   ✔ 90ef3ffc5156 Already exists
[+] Running 4/4
 ✔ Network getting-started-app-main_default            Created
 ✔ Volume "getting-started-app-main_todo-mysql-data"   Created
 ✔ Container getting-started-app-main-mysql-1          Started
 ✔ Container getting-started-app-main-app-1            Started
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

docker ps

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS         PORTS                      NAMES
81c86c760193   node:18-alpine "docker-entrypoint.s…"   8 seconds ago  Up 6 seconds   127.0.0.1:3000->3000/tcp   getting-started-app-main-app-1
216dbd9632b6   mysql:8.0      "docker-entrypoint.s…"   8 seconds ago  Up 6 seconds   3306/tcp, 33060/tcp        getting-started-app-main-mysql-1
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Gestion de plusieurs conteneurs avec Docker Compose

Détruire l'ensemble des ressources d'une stack:

docker compose down

```
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$ docker compose down
[+] Running 3/3
 ✔ Container getting-started-app-main-mysql-1  Removed
 ✔ Container getting-started-app-main-app-1    Removed
 ✔ Network getting-started-app-main_default    Removed
hp@hp-HP-Laptop-15-dw3xxx:~/getting-started-app-main$
```

Conclusion

- Dans ce chapitre
  - Fonctionnement d’un conteneur Docker
  - Création d’une image Docker : fichier Dockerfile
  - Gestion de plusieurs conteneurs avec Docker Compose
- Les conteneurs : **évolutivité, flexibilité** et **facilité**
  - Pour faire évoluer les fonctionnalités de l'application, il suffit alors de déployer de nouveaux conteneurs renfermant les composants appropriés (au lieu de créer entièrement de nouvelles itérations de l'application)

Conclusion

- Les conteneurs : solution idéale pour le développement d'applications de type microservices
  - permettent de configurer des applications complexes à partir de composants élémentaires
  - chacun de ces composants peut être déployé dans un conteneur et les conteneurs constitutifs étant reliés entre eux pour former l'application complète
- Mais, les conteneurs **ne conviennent pas à toutes les applications**
- Ils ne peuvent pas remplacer tous les déploiements de VM existants
- Les conteneurs : <u>ne sont pas adéquats pour les applications monolithiques</u>
- Ces applications peuvent toujours s'exécuter comme des VMs entièrement fonctionnelles adossées à un hyperviseur conventionnel

References

- Cours "Introduction à la virtualisation & Cloud". Jean-Patrick Gelas, UE Cloud Computing – M2, Université Claude Bernard – Lyon
- Documentation officielle Docker - https://docs.docker.com/
- Docker – Components Licences : https://www.docker.com/components-licenses
- Docker – Docker Engine : https://docs.docker.com/engine/understanding-docker/
- https://tropars.github.io/downloads/lectures/DevOps/devops_6_containers.pdf